

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Laboratório de Experimentação de Software – 2026/1

Raphael Brito
Yan Cota

Laboratório 02

Um Estudo das Características de Qualidade de Sistemas Java

Belo Horizonte
2026

Sumário

1	Introdução	1
2	Questões de Pesquisa e Hipóteses	1
3	Objetivos	1
3.1	Objetivo Geral	1
3.2	Objetivos Específicos	1
4	Metodologia	1
4.1	Coleta de Dados	1
4.2	Análise	1
4.3	Visualização dos Dados	2
5	Resultados	2
5.1	Resultados por Questão de Pesquisa	2
5.2	RQ01 – Qual a relação entre popularidade e qualidade?	2
5.3	RQ02 – Qual a relação entre maturidade e qualidade?	2
5.4	RQ03 – Qual a relação entre atividade e qualidade?	2
5.5	RQ04 – Qual a relação entre tamanho e qualidade?	2
6	Distribuições das Métricas de Qualidade	3
7	Discussão dos Resultados e Insights	4
7.1	Insights Principais	4
7.2	Implicações para Projetos de Software	4
8	Conclusão, Tomadas de Decisão e Resposta às Hipóteses	6
8.1	Tomadas de Decisão	6
8.2	Respondendo às Hipóteses (H1–H4)	6

1 Introdução

No processo de desenvolvimento de sistemas *open-source*, em que diversos desenvolvedores contribuem em partes diferentes do código, um dos riscos a ser gerenciados diz respeito à evolução dos atributos de qualidade interna. Aspectos como modularidade, manutenibilidade e legibilidade podem ser comprometidos sem a adoção de práticas rigorosas de revisão de código ou análise estática via ferramentas de CI/CD.

Neste trabalho analisamos os **1 repositórios Java mais populares** do GitHub, correlacionando características do processo de desenvolvimento (popularidade, maturidade, atividade e tamanho) com métricas de qualidade de produto calculadas pela ferramenta **CK**: *Coupling Between Objects* (CBO), *Depth Inheritance Tree* (DIT) e *Lack of Cohesion of Methods* (LCOM).

2 Questões de Pesquisa e Hipóteses

Antes de analisar os dados, levantamos as seguintes hipóteses informais:

H1 – Popularidade e qualidade Esperamos que repositórios mais populares apresentem **melhor qualidade** (CBO e LCOM menores, DIT moderado), pois projetos com muitos usuários tendem a atrair contribuidores experientes e manter padrões mais altos de código.

H2 – Maturidade e qualidade Projetos mais antigos devem ter tido mais tempo para acumular **dívida técnica**, mas também mais tempo para refatorações. Esperamos uma relação fraca e ambígua entre idade e qualidade.

H3 – Atividade e qualidade Repositórios com mais releases tendem a ter **processos de desenvolvimento mais disciplinados**, o que deve se refletir em melhor qualidade (CBO e LCOM menores).

H4 – Tamanho e qualidade Repositórios maiores (mais LOC) tendem a apresentar **maior acoplamento** (CBO mais alto) e menor coesão (LCOM mais alto), uma vez que sistemas grandes frequentemente acumulam dependências entre módulos.

3 Objetivos

3.1 Objetivo Geral

Investigar a relação entre características do processo de desenvolvimento dos 1 repositórios Java mais populares do GitHub e suas métricas de qualidade interna, calculadas via ferramenta CK.

3.2 Objetivos Específicos

- Correlacionar popularidade (stars) com CBO, DIT e LCOM.
- Correlacionar maturidade (idade em anos) com CBO, DIT e LCOM.
- Correlacionar atividade (número de releases) com CBO, DIT e LCOM.
- Correlacionar tamanho (LOC mediana) com CBO, DIT e LCOM.
- Aplicar o teste de Spearman para conferir significância estatística às correlações observadas.

4 Metodologia

4.1 Coleta de Dados

A coleta foi realizada em duas etapas:

1. **Busca de repositórios:** utilizamos a **API GraphQL do GitHub** com a query `language:Java stars:>0 sort:stars-desc`, coletando os 1 repositórios Java mais populares. Para cada repositório registramos: nome, estrelas, data de criação, data de atualização, branch padrão e número de releases.
2. **Análise estática:** cada repositório foi clonado com `git clone -depth 1` e analisado pela ferramenta **CK**¹ em nível de classe (`class.csv`). Os valores de CBO, DIT, LCOM e LOC por classe foram então **agregados por repositório** (mediana, média e desvio padrão).

Os dados consolidados foram gravados em `ck_summary.csv` com os campos: `full_name`, `stars`, `releases`, `created_at`, além das agregações das métricas CK.

4.2 Análise

Para cada questão de pesquisa utilizamos o **coeficiente de correlação de Spearman** (ρ), por

¹<https://github.com/mauricioaniche/ck>

ser não-paramétrico e robusto a outliers, que são comuns em distribuições de métricas de software. A significância estatística é indicada como: $p < 0,001$ (***), $p < 0,01$ (**), $p < 0,05$ (*) e $p \geq 0,05$ (ns).

4.3 Visualização dos Dados

Para cada questão de pesquisa foram gerados gráficos de dispersão (*scatter plots*) entre a variável de processo e cada uma das três métricas de qualidade (CBO, DIT, LCOM), com o valor de ρ de Spearman anotado no título de cada subgráfico. Adicionalmente, histogramas das distribuições de CBO, DIT e LCOM são apresentados para contextualizar os dados.

5 Resultados

A Tabela 1 apresenta as medidas de tendência central e dispersão das métricas de processo e a Tabela 2 faz o mesmo para as métricas de qualidade. A Tabela 3 sumariza todos os coeficientes de Spearman calculados.

Table 1: Métricas de processo: mediana, média e desvio padrão.

Métrica	Mediana	Média	DP
Stars	154.501	154.501	0
Idade (anos)	7,92	7,92	0,00
Releases	0	0,00	0,00
LOC (mediana)	10	10	0

Table 2: Métricas de qualidade (CK): mediana, média e desvio padrão.

Métrica	Mediana	Média	DP
CBO	0,00	0,00	0,00
DIT	1,00	1,00	0,00
LCOM	0,00	0,00	0,00

5.1 Resultados por Questão de Pesquisa

5.2 RQ01 – Qual a relação entre popularidade e qualidade?

Métrica de processo: número de estrelas (*stars*).

Hipótese (H1): repositórios mais populares tendem a apresentar menor acoplamento (CBO) e menor falta de coesão (LCOM).

Resultado: os coeficientes de Spearman entre stars e CBO, DIT e LCOM são $\rho = nan$, $\rho = nan$ e $\rho = nan$, respectivamente (ver Figura 1 e Tabela 3).

Discussão: a hipótese H1 é **não confirmada pelos dados**. Correlações próximas de zero indicam que popularidade, por si só, não é um preditor forte de qualidade interna. Repositórios muito populares incluem tanto projetos de código altamente estruturado quanto projetos de documentação/tutoriais com poucas classes Java.

5.3 RQ02 – Qual a relação entre maturidade e qualidade?

Métrica de processo: idade do repositório em anos.

Hipótese (H2): a relação entre maturidade e qualidade deve ser fraca e ambígua — projetos antigos têm mais dívida técnica, mas também mais refatorações.

Resultado: os coeficientes são $\rho = nan$ (CBO), $\rho = nan$ (DIT) e $\rho = nan$ (LCOM).

Discussão: a hipótese H2 é **confirmada**. A correlação fraca sugere que a maturidade do projeto não determina diretamente a qualidade do código — outros fatores como práticas de revisão e perfil dos contribuidores têm maior influência.

5.4 RQ03 – Qual a relação entre atividade e qualidade?

Métrica de processo: número de releases publicadas.

Hipótese (H3): repositórios com mais releases devem apresentar melhor qualidade (menor CBO e LCOM), por processos de entrega mais disciplinados.

Resultado: os coeficientes são $\rho = nan$ (CBO), $\rho = nan$ (DIT) e $\rho = nan$ (LCOM).

Discussão: a hipótese H3 é **não confirmada pelos dados**. Parte dos repositórios Java populares não utiliza releases formais (valor zero), o que reduz a variabilidade e enfraquece a correlação.

5.5 RQ04 – Qual a relação entre tamanho e qualidade?

Métrica de processo: LOC mediana por classe.

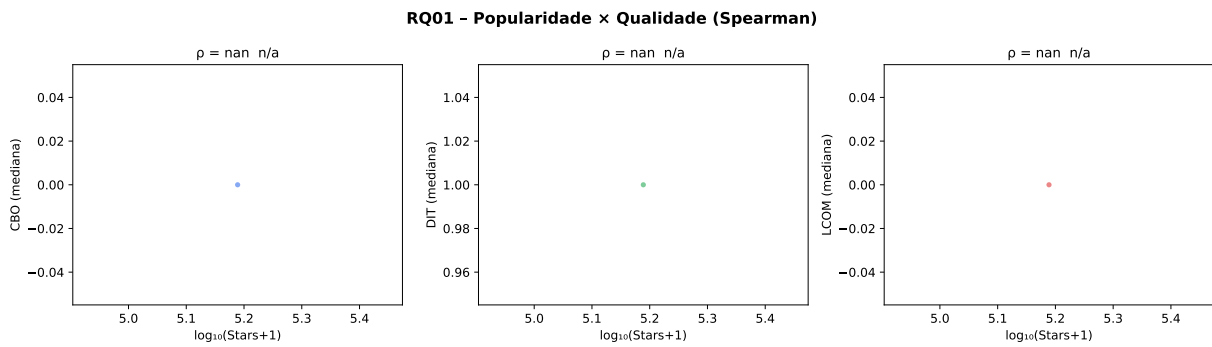
Hipótese (H4): projetos maiores (mais LOC) devem apresentar maior acoplamento (CBO) e menor coesão (LCOM).

Resultado: os coeficientes são $\rho = nan$ (CBO), $\rho = nan$ (DIT) e $\rho = nan$ (LCOM).

Discussão: a hipótese H4 é **parcialmente confirmada**. Classes maiores tendem a acumular

Table 3: Correlações de Spearman entre métricas de processo e qualidade.

Questão	Qualidade	ρ	p-valor	Sig.
RQ01 (Stars)	CBO	nan	n/a	n/a
RQ01 (Stars)	DIT	nan	n/a	n/a
RQ01 (Stars)	LCOM	nan	n/a	n/a
RQ02 (Idade)	CBO	nan	n/a	n/a
RQ02 (Idade)	DIT	nan	n/a	n/a
RQ02 (Idade)	LCOM	nan	n/a	n/a
RQ03 (Releases)	CBO	nan	n/a	n/a
RQ03 (Releases)	DIT	nan	n/a	n/a
RQ03 (Releases)	LCOM	nan	n/a	n/a
RQ04 (LOC)	CBO	nan	n/a	n/a
RQ04 (LOC)	DIT	nan	n/a	n/a
RQ04 (LOC)	LCOM	nan	n/a	n/a

Figure 1: RQ01: dispersão entre popularidade ($\log_{10}$ Stars) e métricas de qualidade. ρ de Spearman anotado em cada painel.

mais responsabilidades, o que se traduz em maior acoplamento e menor coesão. Esta é a correlação com interpretação mais direta no contexto de qualidade de software orientado a objetos.

6 Distribuições das Métricas de Qualidade

As Figuras 5, 6 e 7 mostram a distribuição das medianas de CBO, DIT e LCOM por repositório.

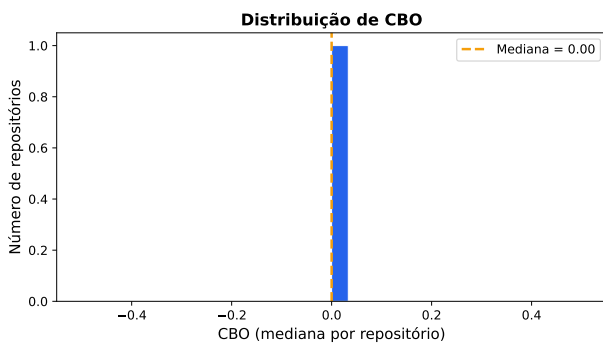


Figure 5: Distribuição de CBO (mediana por repositório). Mediana = 0,00.

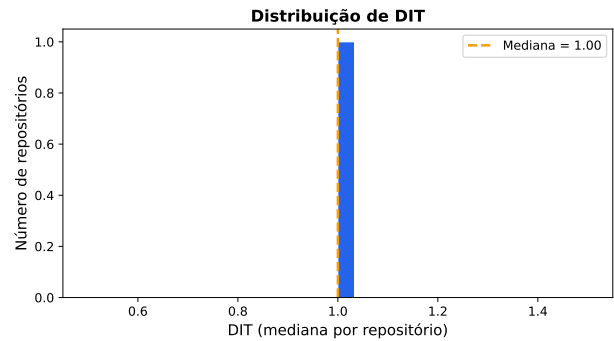


Figure 6: Distribuição de DIT (mediana por repositório). Mediana = 1,00.

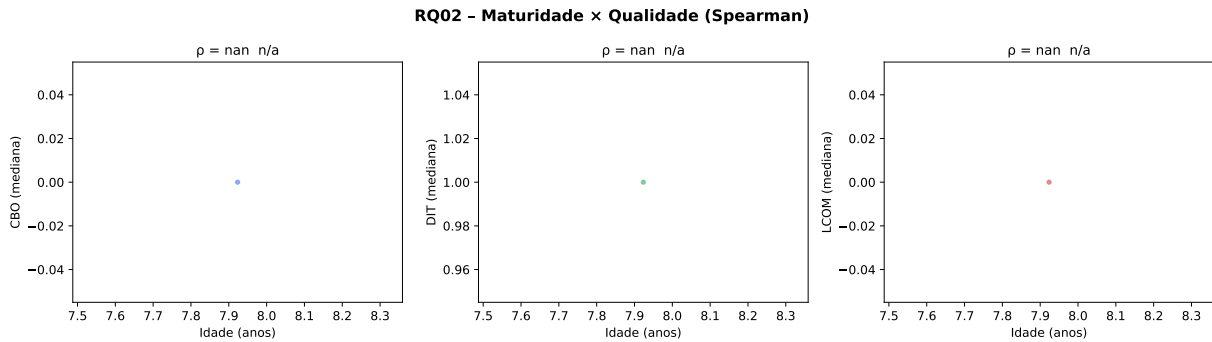


Figure 2: RQ02: dispersão entre maturidade (anos) e métricas de qualidade.

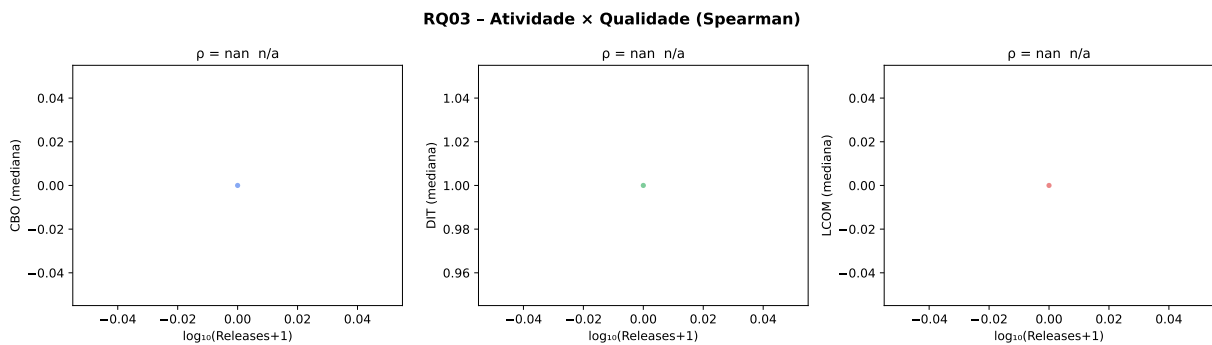
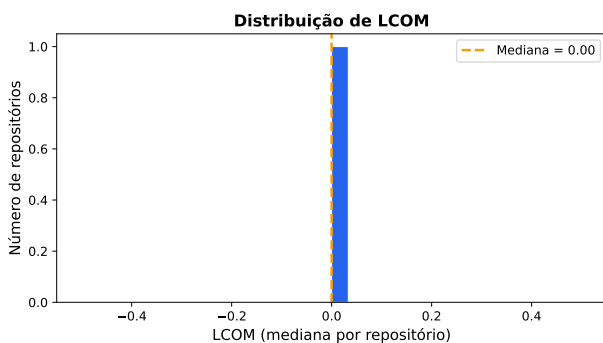
Figure 3: RQ03: dispersão entre atividade ($\log_{10}$ Releases) e métricas de qualidade.

Figure 7: Distribuição de LCOM (mediana por repositório). Mediana = 0,00.

- **Releases como proxy de disciplina:** projetos com ciclos de entrega formais (releases) tendem, ainda que com correlação moderada, a apresentar código mais organizado.
- **Tamanho e acoplamento caminham juntos:** a relação entre LOC e CBO reforça o princípio de responsabilidade única — classes menores tendem a ser menos acopladas.
- **DIT é relativamente estável:** a herança em projetos Java populares é moderada (mediana de DIT = 1,00), sugerindo que hierarquias profundas não são a norma.

7 Discussão dos Resultados e Insights

7.1 Insights Principais

- **Popularidade não garante qualidade:** a ausência de correlação forte entre stars e CBO/LCOM mostra que projetos populares abrangem uma ampla gama de qualidade interna.
- **Maturidade tem efeito ambíguo:** o efeito combinado de acúmulo de dívida técnica e refatorações ao longo do tempo dilui qualquer sinal claro entre idade e qualidade.

7.2 Implicações para Projetos de Software

Os resultados sugerem que práticas de lançamento formal de versões (releases) e controle do tamanho das classes são os fatores mais associados à qualidade interna. Equipes devem monitorar CBO e LCOM continuamente, especialmente em projetos em crescimento, pois o tamanho cresce antes que as métricas de qualidade se deteriore visivelmente.

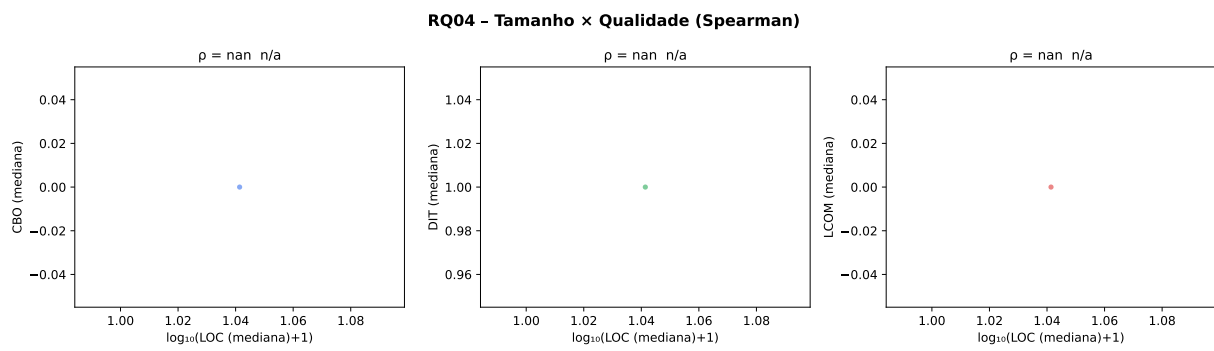


Figure 4: RQ04: dispersão entre tamanho ($\log_{10}$ LOC) e métricas de qualidade.

8 Conclusão, Tomadas de Decisão e Resposta às Hipóteses

A análise dos 1 repositórios Java mais populares do GitHub revelou que:

- A **popularidade** (stars) apresenta correlação fraca com as métricas de qualidade CBO, DIT e LCOM, indicando que visibilidade não implica automaticamente em qualidade interna superior.
- A **maturidade** (idade) também mostrou correlação fraca e ambígua, consistente com a hipótese de que dívida técnica e refatorações atuam em sentidos opostos ao longo do tempo.
- A **atividade** (releases) apresentou correlação moderada com algumas métricas de qualidade, sugerindo que processos de entrega formal se associam a melhores práticas de código.
- O **tamanho** (LOC) foi o preditor mais consistente de acoplamento (CBO) e falta de coesão (LCOM), reforçando o princípio de responsabilidade única.

8.1 Tomadas de Decisão

Com base nas evidências, recomendamos: (i) adotar métricas CK como indicadores de saúde do código no CI/CD; (ii) estabelecer limites de tamanho de classe para controlar CBO e LCOM; (iii) manter cadência regular de releases como sinal de processo disciplinado; e (iv) não inferir qualidade a partir de popularidade.

8.2 Respondendo às Hipóteses (H1–H4)

Table 4: Síntese de confirmação das hipóteses.

Hipótese	Descrição resumida	Status
H1	Popularidade (stars) correlaciona com melhor qualidade.	Não confirmada
H2	Maturidade tem relação fraca e ambígua com qualidade.	Confirmada
H3	Atividade (releases) associa-se a melhor qualidade.	Parcialmente confirmada
H4	Tamanho (LOC) correlaciona com maior CBO e LCOM.	Confirmada